INFORMATIK LK · WELT 4 · DS 24–30

Der Hafen

Stapel, Schlange, dynamische Reihung



L4.1 · L4.2 · L4.3 · L4.4 · L4.5 · L4.6 · BOSS

Der Hafen im Überblick



DS 24

IDE-Level 4.1

Stapel und Schlange

LIFO, FIFO, Casten, Leer-Abfrage

Lab 4.1



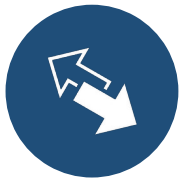
DS 25 · Kür

IDE-Level 4.2

Spielmodule

Undo/Redo, Initiative, Eignung

Lab 4.2



DS 26

IDE-Level 4.3

Unverändert

Hilfsstapel, Hilfs-schlange, Rotieren

Lab 4.3



DS 27

IDE-Level 4.4

Ladeliste

DynArray, sortiert einfügen, löschen

Lab 4.4



DS 28

IDE-Level 4.5

Umformen

Klammerprüfung, Postfix-Rechner

Lab 4.5



DS 29 · Kür

IDE-Level 4.6

Expertenpfad

Flood-Fill iterativ, eigener Stapel

Lab 4.6



DS 30

Boss 4

Nebelhafen

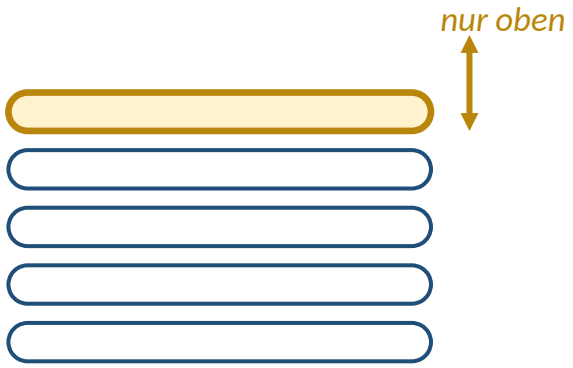
Abi auf Zeit: 24 BE in 60 min

+ Online-Boss

Roter Faden: Die Struktur bestimmt, wer als Nächstes drankommt – und vor jedem Zugriff fragst du: Leer? Index?

Datenstrukturen im Alltag: Wer kommt als Nächstes dran?

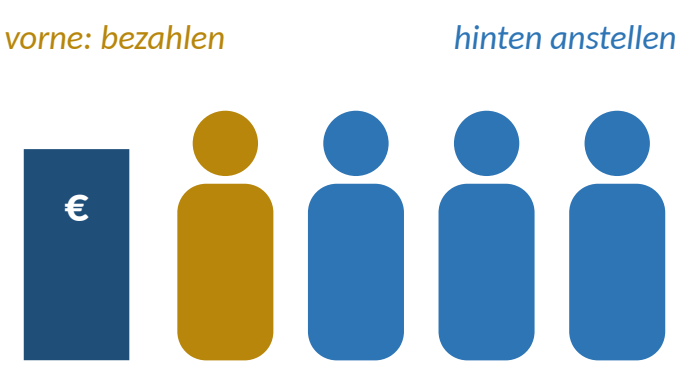
Tellerstapel



Stapel · LIFO

Wer zuletzt draufgelegt wurde, wird zuerst genommen.

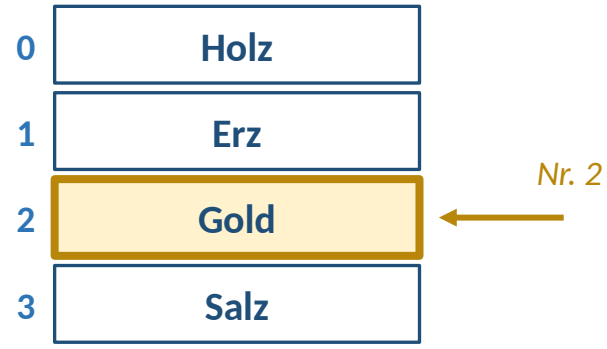
Kasse im Hafenladen



Schlange · FIFO

Wer sich zuerst angestellt hat, zahlt zuerst.

Ladeliste

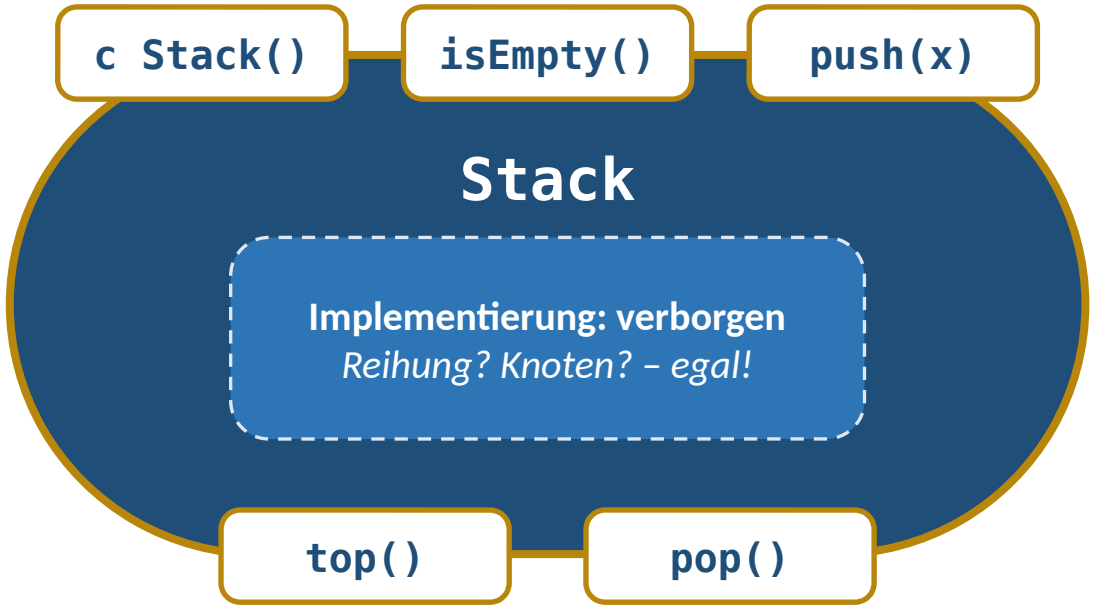


dynamische Reihung

Zugriff auf jede Zeile über ihre Nummer – einfügen und streichen überall.

Leitfrage: Wo darf man hineinlegen, wo herausnehmen? – Das legt die Struktur fest, nicht der Inhalt.

Abstrakter Datentyp: Man sieht nur die Schnittstelle



Schnittstelle: nur diese Operationen sind von außen benutzbar

Abstrakter Datentyp (ADT): Daten und Operationen, beschrieben nur durch die Schnittstelle – nicht durch die Implementierung.

Kapselung
An die Daten kommst du nur über die Operationen.

Austauschbar
Innen eine Reihung oder Knoten (DS 29) – dein Programm merkt es nicht.

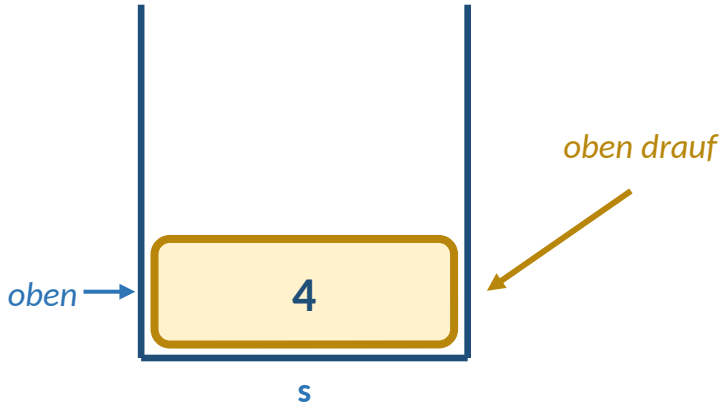
Abitur (EH §4)
Stapel, Schlange, dynamische Reihung, Binärbaum mit festen Operationen.

Merke: Du programmierst gegen die Schnittstelle – genau diese Operationen darfst du benutzen, sonst keine.

Stapel und Schlange: Wer kommt zuerst heraus?

Stapel s · LIFO: Last In, First Out

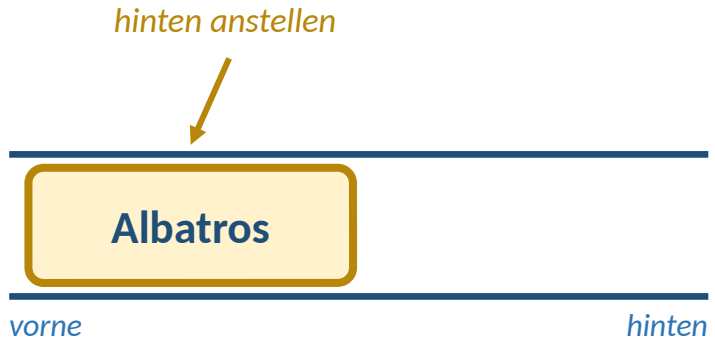
```
Stack s = new Stack();
s.push(4);
s.push(8);
int x = (Integer) s.pop();
```



push(4): 4 wird oben aufgelegt – der Stapel war leer.

Schlange q · FIFO: First In, First Out

```
Queue q = new Queue();
q.enqueue("Albatros");
q.enqueue("Moewe");
String n = (String) q.dequeue();
```

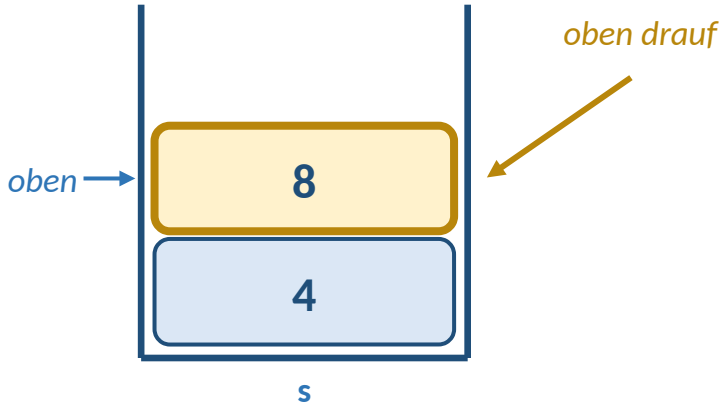


enqueue("Albatros"): stellt sich hinten an – und steht damit auch vorne.

Stapel und Schlange: Wer kommt zuerst heraus?

Stapel s · LIFO: Last In, First Out

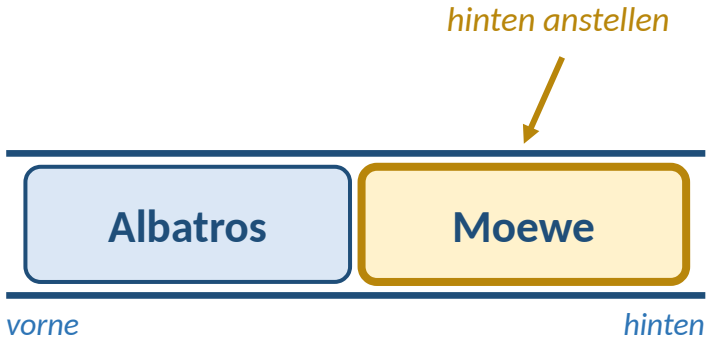
```
Stack s = new Stack();
s.push(4);
s.push(8);
int x = (Integer) s.pop();
```



push(8): 8 liegt jetzt oben auf der 4.

Schlange q · FIFO: First In, First Out

```
Queue q = new Queue();
q.enqueue("Albatros");
q.enqueue("Moewe");
String n = (String) q.dequeue();
```

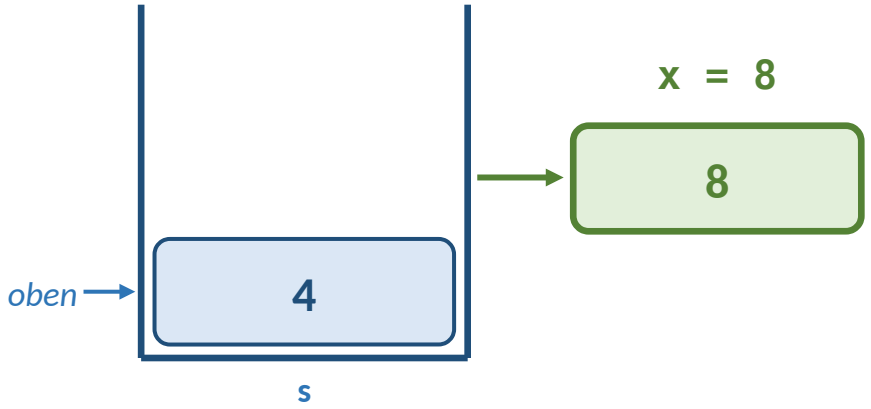


enqueue("Moewe"): Moewe stellt sich hinter Albatros an.

Stapel und Schlange: Wer kommt zuerst heraus?

Stapel s · LIFO: Last In, First Out

```
Stack s = new Stack();
s.push(4);
s.push(8);
int x = (Integer) s.pop();
```



pop() entfernt das OBERSTE Element und liefert 8 – das zuletzt hineingelegte (LIFO).

Schlange q · FIFO: First In, First Out

```
Queue q = new Queue();
q.enqueue("Albatros");
q.enqueue("Moewe");
String n = (String) q.dequeue();
```



dequeue() entfernt das VORDERSTE Element und liefert "Albatros" – das zuerst angestellte (FIFO).

Die Schnittstellen nach EH §4

Stapel – Klasse Stack

Operation	Wirkung	leer?
`c Stack()`	erzeugt einen leeren Stapel	
`isEmpty(): Wahrheitswert`	wahr, wenn der Stapel leer ist	
`push(x: Inhaltstyp): nichts`	legt x oben auf	
`top(): Inhaltstyp`	liefert das oberste Element, ohne es zu entfernen	**Fehler**
`pop(): Inhaltstyp`	entfernt das oberste Element und liefert es	**Fehler**

Schlange – Klasse Queue

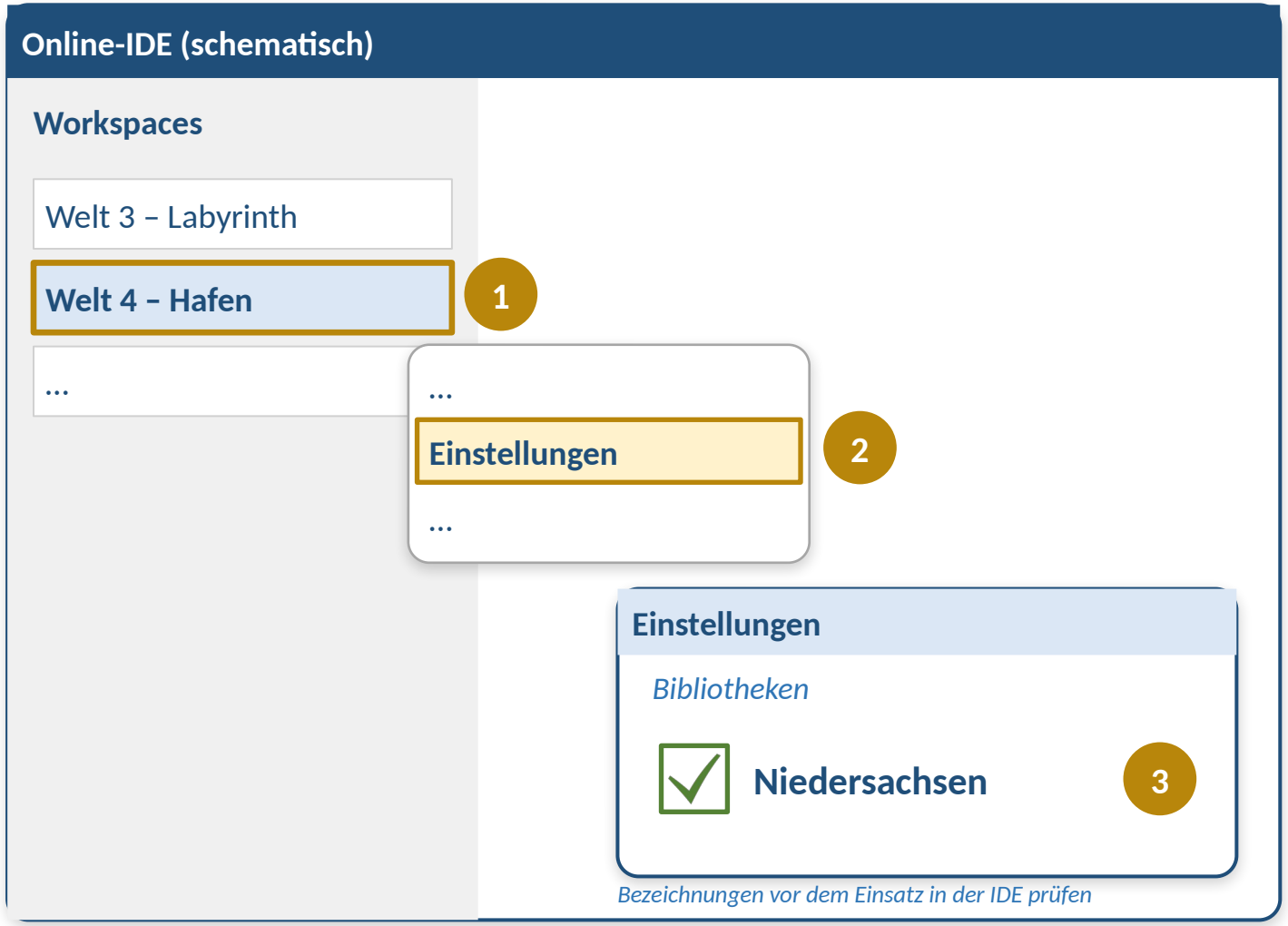
Operation	Wirkung	leer?
`c Queue()`	erzeugt eine leere Schlange	
`isEmpty(): Wahrheitswert`	wahr, wenn die Schlange leer ist	
`enqueue(x: Inhaltstyp): nichts`	stellt x hinten an	
`head(): Inhaltstyp`	liefert das vorderste Element, ohne es zu entfernen	**Fehler**
`dequeue(): Inhaltstyp`	entfernt das vorderste Element und liefert es	**Fehler**

In Signaturen:

```
s: Stapel vom Inhaltstyp Kiste
q: Schlange vom Inhaltstyp Ganzzahl
```

Leer? top, pop, head und dequeue auf einer leeren Struktur führen zum Laufzeitfehler. Vorher isEmpty() fragen! push(null) und enqueue(null) sind verboten.

Bibliothek „Niedersachsen“ einschalten



1. Workspace mit der rechten Maustaste anklicken
2. „Einstellungen“ wählen
3. Häkchen bei „Niedersachsen“ setzen

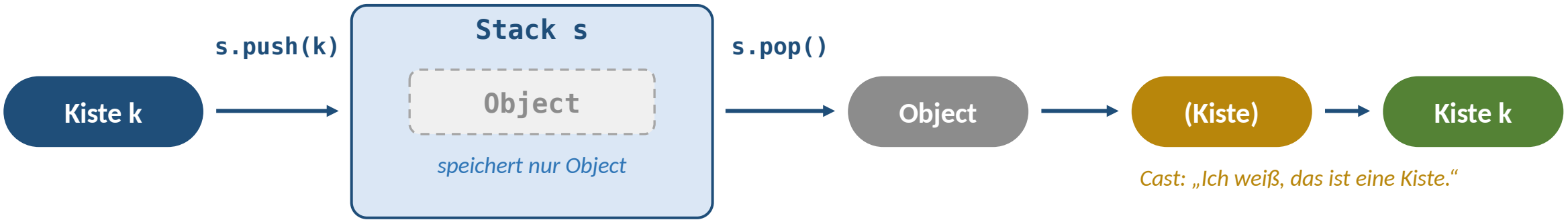
Danach bekannt:

- Stack
- Queue
- DynArray
- BinTree

Ausgeblendet (java.util): Stack, Queue, Deque, LinkedList, ArrayList, Vector

Ohne Häkchen: „Stack“ unbekannt – nichts kompiliert. Einmal pro Workspace!

Casten bei Abitur-Klassen: Object hinein, Kiste heraus




```
Stack s = new Stack();            // keine spitzen Klammern!
Kiste k = (Kiste) s.pop();        // Stapel vom Inhaltstyp Kiste
int x = (Integer) q.dequeue();    // Schlange vom Inhaltstyp Ganzzahl
char c = (Character) s.pop();     // Stapel vom Inhaltstyp Zeichen
String n = (String) q.head();     // Schlange vom Inhaltstyp Zeichenkette
Kiste k = (Kiste) d.getItem(i);   // dynamische Reihung vom Inhaltstyp Kiste
```

Hineinlegen ohne Cast: `s.push(5)`, `q.enqueue("Moewe")` – Zahlen und Zeichen werden automatisch verpackt.

Cast vergessen: Fehler schon beim Kompilieren – Object passt nicht zu Kiste.

Falscher Cast: Laufzeitfehler ClassCastException, z. B. (Integer) auf eine Kiste.

Papier-Compiler: Casts auch handschriftlich mitschreiben!

Laufzeitfehler vermeiden: Erst fragen, dann nehmen

falsch *obersterWert: leerer Stapel → −1*

```
public static int obersterWert(Stack s) {
    return (Integer) s.top();   // leer? Absturz!
}
```

richtig

```
public static int obersterWert(Stack s) {
    if (s.isEmpty()) {
        return -1;   // Leer? -> kein top()!
    }
    return (Integer) s.top();
}
```

Laufzeitfehler	Ursache
IllegalStateException	top, pop, head, dequeue auf leerer Struktur
IndexOutOfBoundsException	DynArray-Index nicht in 0 … getLength() − 1
IllegalArgumentException	null hineinlegen
ClassCastException	falscher Cast

Abi-Check Welt 4

- ✓ Leer? – vor top, pop, head, dequeue: isEmpty()
- ✓ Index? – 0 ≤ i < getLength()
- ✓ Cast beim Entnehmen
- ✓ unverändert, falls gefordert

Achtung Abi: Laufzeitfehler müssen durch Abfragen ausgeschlossen werden (EH §4) – auch bei leeren Strukturen.

Mischaufgabe: Stapel und Schlange zusammen tracen

```
Queue q = ...;    // q: 3 1 4 1 5 (vorne 3)
Stack s = new Stack();
1  s.push(q.dequeue());
2  s.push(q.dequeue());
3  q.enqueue(s.pop());
4  q.enqueue(q.dequeue());
5  s.push(q.head());
6  int x = (Integer) s.pop()
         + (Integer) q.dequeue();
7  s.push(x);
```

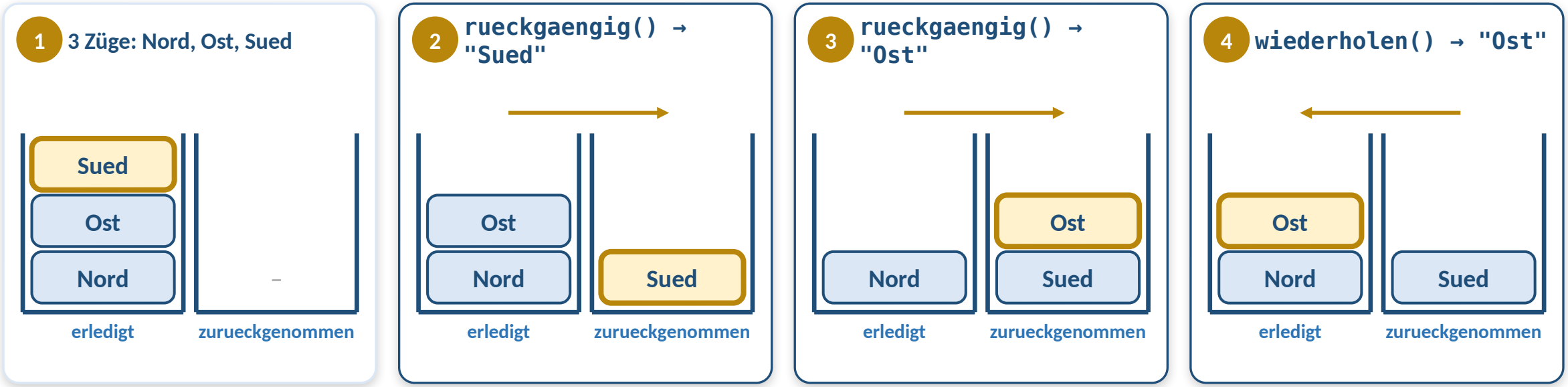
Fehlerquellen: Zeile 4 – das vordere Element wandert nach hinten. Zeile 5 – head() entnimmt nicht: q bleibt gleich!

Notation: Stapel von oben, Schlange von vorne, – = leer

Nr	q (vorne … hinten)	s (oben … unten)	Bemerkung
0	3 1 4 1 5	–	Start
1	1 4 1 5	3	3 auf s
2	4 1 5	1 3	1 liegt oben
3	4 1 5 1	3	1 hinten angestellt
4	1 5 1 4	3	rotiert: 4 nach hinten
5	1 5 1 4	1 3	head: q bleibt!
6	5 1 4	3	x = 1 + 1 = 2
7	5 1 4	2 3	x aufgelegt

Am Ende: q = 5 1 4 · s = 2 3 · x = 2

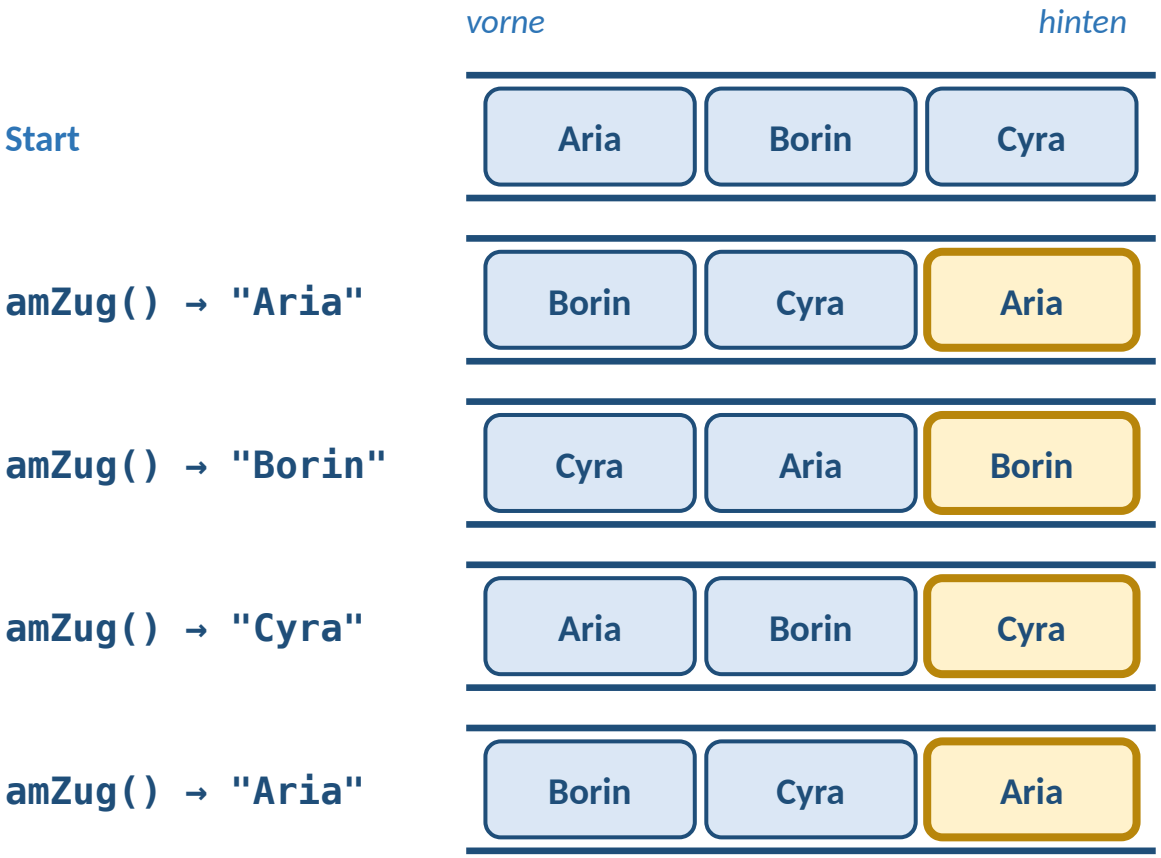
Undo/Redo im Zugbuch: zwei Stapel




```
public String rueckgaengig() {
    if (erledigt.isEmpty()) {
        return "";        // Leer? -> kein pop()!
    }
    String z = (String) erledigt.pop();
    zurueckgenommen.push(z);
    return z;
}
```

`zug(z)`	push auf erledigt – und zurueckgenommen wird geleert
`rueckgaengig()`	oberster Zug: erledigt → zurueckgenommen
`wiederholen()`	oberster Zug: zurueckgenommen → erledigt

Initiative: Die Zugreihenfolge rotiert in einer Schlange



Reihenfolge: Aria, Borin, Cyra, Aria, ...

```java
public String amZug() {
    if (reihe.isEmpty()) {
        return "";
    }
    String name = (String) reihe.dequeue();
    reihe.enqueue(name);   // hinten anstellen
    return name;
}
```

Rotieren: `q.enqueue(q.dequeue())` – nach n-mal Rotieren (n Elemente) ist die Schlange wieder wie vorher.

Achtung: Rotieren, „bis der Name vorne steht“, ohne Zähler: Fehlt der Name, dreht sich die Schlange endlos. Erst die Länge bestimmen – oder eine Hilfsschlange benutzen.

Welche Struktur passt? Eignung beurteilen

	Stapel	Schlange	dynamische Reihung
Zugriff	nur oben: top, pop	vorne entnehmen, hinten anstellen	überall über den Index
Prinzip	LIFO – das Neueste zuerst	FIFO – das Älteste zuerst	Position frei wählbar
im Spiel	Zugbuch: Rückgängig	Initiative: Zugreihenfolge	Inventar, Ladeliste
Schwäche	an untere Elemente nur durch Umschichten	an mittlere nur durch Rotieren	Einfügen und Löschen verschieben die Nachfolger

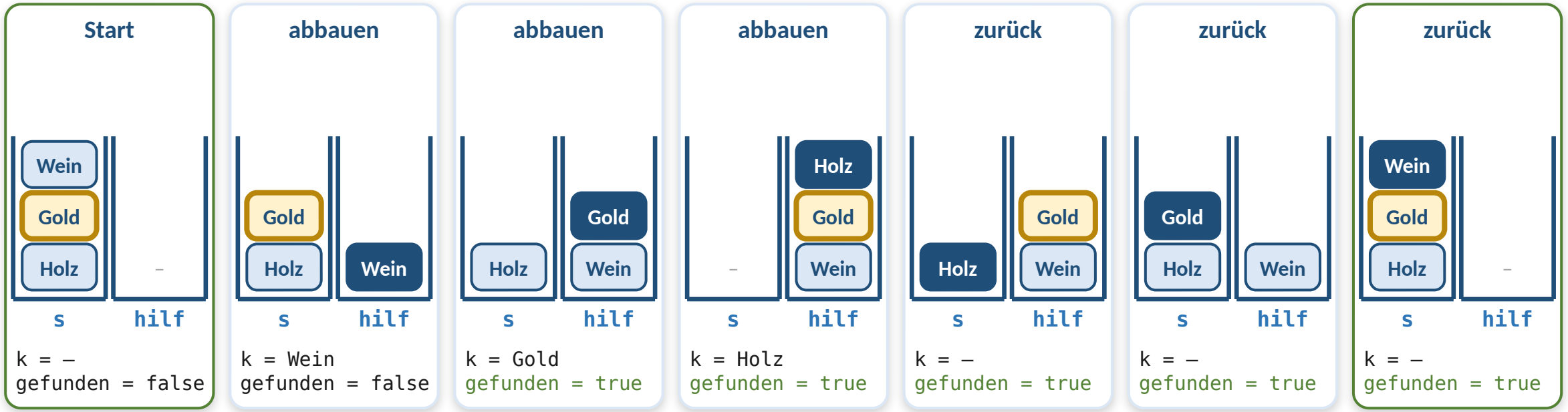
So beurteilst du im Abi

1. Struktur nennen
2. Zugriffsprinzip angeben (LIFO, FIFO, Index)
3. auf den Kontext beziehen
4. Alternative begründet ausschließen

Beispiel: Funksprüche im Hafenbüro werden in der Reihenfolge ihres Eingangs beantwortet.

Musterantwort: Eine Schlange: Neue Funksprüche werden hinten angestellt, beantwortet wird vorne – der älteste zuerst (FIFO). Ein Stapel lieferte den neuesten zuerst.

Muster „unverändert lassen“: der Hilfsstapel



Phase 1: pop von s · auswerten · push auf hilf

Phase 2: pop von hilf · push auf s

`enthaelt(s, "Gold")` liefert true – s ist danach wieder Wein, Gold, Holz: dieselben Kisten in derselben Reihenfolge.

Achtung: Mit return true sofort beim Fund bliebe s = Holz – Wein und Gold lägen noch im Hilfsstapel!

Das Muster als Struktogramm – und bei der Schlange

auswerten(s: Stapel vom Inhaltstyp Kiste): …
hilf ← erzeuge Stapel
solange nicht s.isEmpty()
▸ k ← s.pop() *// Cast (Kiste) in Java*
▸ *// … k auswerten …*
▸ hilf.push(k) *// dieselbe Kiste aufheben*
solange nicht hilf.isEmpty()
▸ s.push(hilf.pop()) *// zurückschichten*
gib … zurück

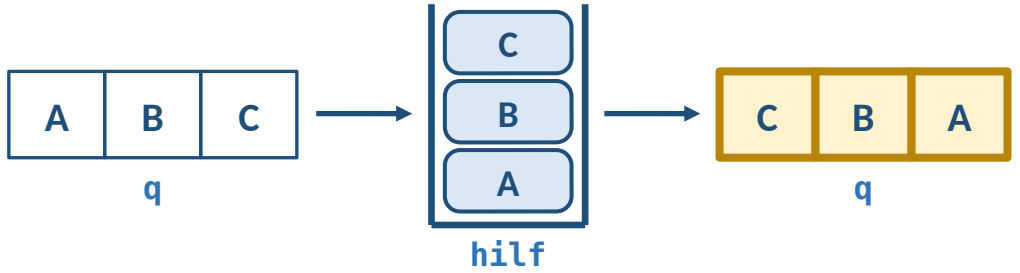
Stapel → Hilfsstapel → Stapel: zweimal umgedreht = alte Reihenfolge.

unverändert = dieselben Objekte, nicht new Kiste(…) mit gleichen Werten.

Und bei der Schlange?

Weg	Reihenfolge danach
`Schlange → Hilfsschlange → Schlange`	**bleibt ✓**
`n-mal q.enqueue(q.dequeue())`	**bleibt ✓ (n bekannt)**
`Schlange → Hilfsstapel → Schlange`	**umgekehrt – richtig nur für umkehren**

umkehren(q) mit Hilfsstapel



summe(4 8 15 16 23 42) = 108 · q bleibt gleich

Die dynamische Reihung (DynArray) nach EH §4

Operation	Wirkung
`c DynArray()`	erzeugt eine leere dynamische Reihung
`isEmpty(): Wahrheitswert`	wahr, wenn leer
`getLength(): Ganzzahl`	Anzahl der Elemente
`getItem(i): Inhaltstyp`	Element an Index i
`setItem(i, x): nichts`	ersetzt das Element an Index i
`append(x): nichts`	hängt x hinten an
`insertAt(i, x): nichts`	fügt x an Index i ein – Nachfolger rücken auf
`delete(i): nichts`	entfernt das Element an Index i – Nachfolger rücken nach

Index? getItem, setItem, delete: 0 ≤ i < getLength() · insertAt: 0 ≤ i ≤ getLength()

d – Ladeliste



`d.insertAt(1, Blei)` – *Erz, Gold, Salz rücken auf*



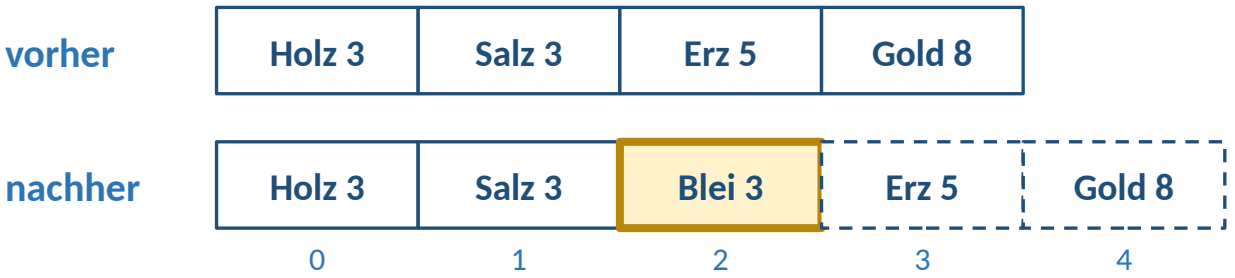
`d.delete(2)` – *Erz weg, Gold und Salz rücken nach*



Merke: Nach insertAt und delete haben alle Nachfolger einen neuen Index.

Sortiert einfügen: einfuegenSortiert(d, Blei 3)

```
einfuegenSortiert(d: dynamische Reihung vom Inhaltstyp Kiste, k: Kiste): nichts
pos ← 0
gefunden ← falsch
solange pos < d.getLength() und nicht gefunden
    x ← d.getItem(pos)
    x.getGewicht() > k.getGewicht()
    wahr                                  falsch
    gefunden ← wahr                       pos ← pos + 1
d.insertAt(pos, k)  // pos = d.getLength(): hängt an
```



pos	getItem(pos)	Aktion
0	Holz 3	pos = 1
1	Salz 3	pos = 2
2	Erz 5	**gefunden → insertAt(2, Blei)**

Hinter gleich schwere: Vergleich mit > (nicht ≥) – Blei kommt hinter Holz und Salz.

Anker 9 kg: Die Schleife endet mit pos = 4 = getLength() – insertAt(4, Anker) hängt hinten an. Kein Sonderfall nötig.

Fehlerbild: delete in der Vorwärtsschleife

FALSCH: i++ auch nach delete

abspalten(d, 5) · d = Holz 3, Gold 8, Tuch 8, Erz 5

```
for (int i = 0; i < d.getLength(); i++) {
    Kiste k = (Kiste) d.getItem(i);
    if (k.getGewicht() > grenze) {
        schwer.append(k);
        d.delete(i);  // Nachfolger rueckt auf i
    }                 // → Tabellenzeile, dann i++
}
```

i	Kiste	Aktion	d danach
0	Holz 3	i = 1	Holz Gold Tuch Erz
1	Gold 8	delete(1), i = 2	Holz Tuch Erz
2	Erz 5	i = 3	Holz Tuch Erz

Tuch übersprungen! schwer = Gold

RICHTIG: i nur im else erhöhen

schwer: alle Kisten mit Gewicht > 5

```
int i = 0;
while (i < d.getLength()) {
    Kiste k = (Kiste) d.getItem(i);
    if (k.getGewicht() > grenze) {
        schwer.append(k);
        d.delete(i);  // i NICHT erhoehen
    } else {
        i++;
    }   // → Tabellenzeile
}
```

i	Kiste	Aktion	d danach
0	Holz 3	i = 1	Holz Gold Tuch Erz
1	Gold 8	delete(1)	Holz Tuch Erz
1	Tuch 8	delete(1)	Holz Erz
1	Erz 5	i = 2	Holz Erz

schwer = Gold, Tuch · d = Holz, Erz

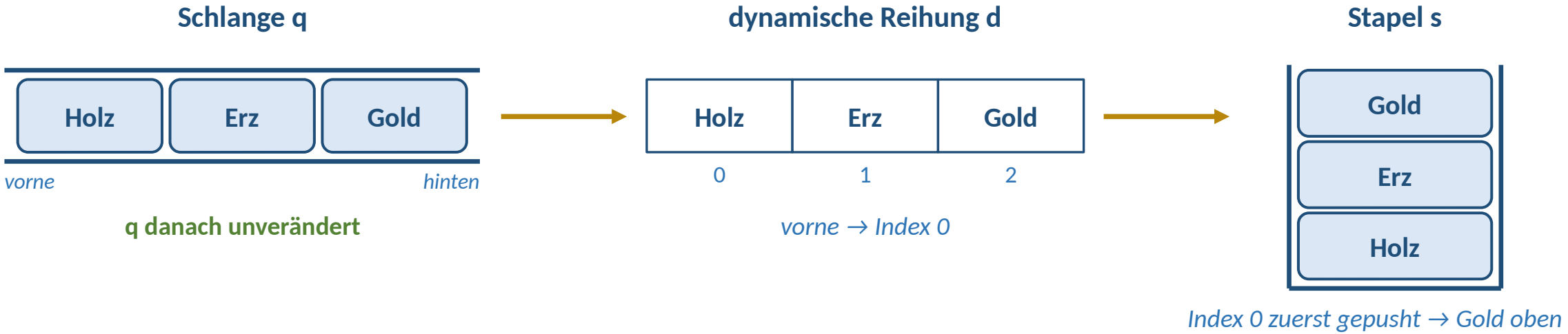
Reihung, DynArray, Stapel, Schlange im Vergleich

	Reihung	dynamische Reihung	Stapel	Schlange
Größe	fest bei der Erzeugung	wächst und schrumpft	beliebig viele Elemente	beliebig viele Elemente
Zugriff	r[i] an jeder Stelle	getItem(i) an jeder Stelle	nur oben: top, pop	nur vorne: head, dequeue
Hinzufügen	nicht möglich, nur überschreiben	append, insertAt – Nachfolger rücken auf	push: oben	enqueue: hinten
Entfernen	nicht möglich	delete(i) – Nachfolger rücken nach	pop: oben	dequeue: vorne
Vor dem Zugriff	0 ≤ i < r.length	0 ≤ i < getLength()	isEmpty()	isEmpty()
Java	`int[] r = new int[5];`	`DynArray d = new DynArray();`	`Stack s = new Stack();`	`Queue q = new Queue();`
Signatur	Reihung von Kiste	dynamische Reihung vom Inhaltstyp Kiste	Stapel vom Inhaltstyp Kiste	Schlange vom Inhaltstyp Kiste

Faustregel: Reihenfolge des Hereinkommens entscheidet → Stapel oder Schlange. Zugriff über die Position → (dynamische) Reihung.

Umformen zwischen den Strukturen

`schlangeZuDynArray(q)` `dynArrayZuStapel(d)`



q danach unverändert

vorne → Index 0

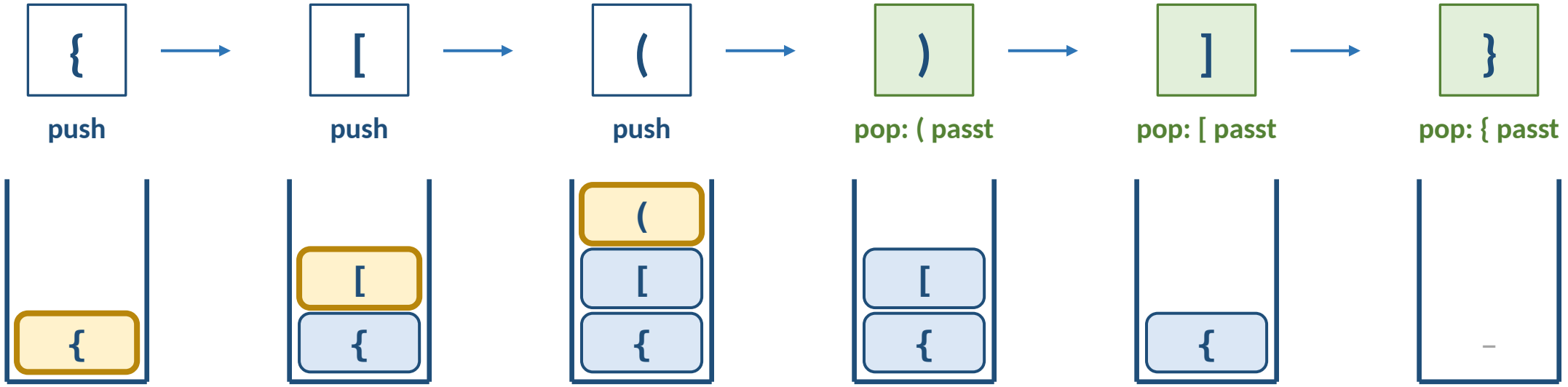
Index 0 zuerst gepusht → Gold oben

```
Stack s = new Stack();
for (int i = 0; i < d.getLength(); i++) {
    s.push(d.getItem(i));   // Object rein: kein Cast
}
```

Achtung: Wer die Schlange abbaut, muss sie danach wiederherstellen.

Merke: Einmal über einen Stapel dreht die Reihenfolge um.

Klammerprüfung mit Stapel: "{[()]}" Schritt für Schritt



Stapel vom Inhaltstyp Zeichen: `offen.push(c); char auf = (Character) offen.pop();`

Ende: Stapel leer → true

"(]"

pop liefert (, aber] braucht [– Art passt nicht → false

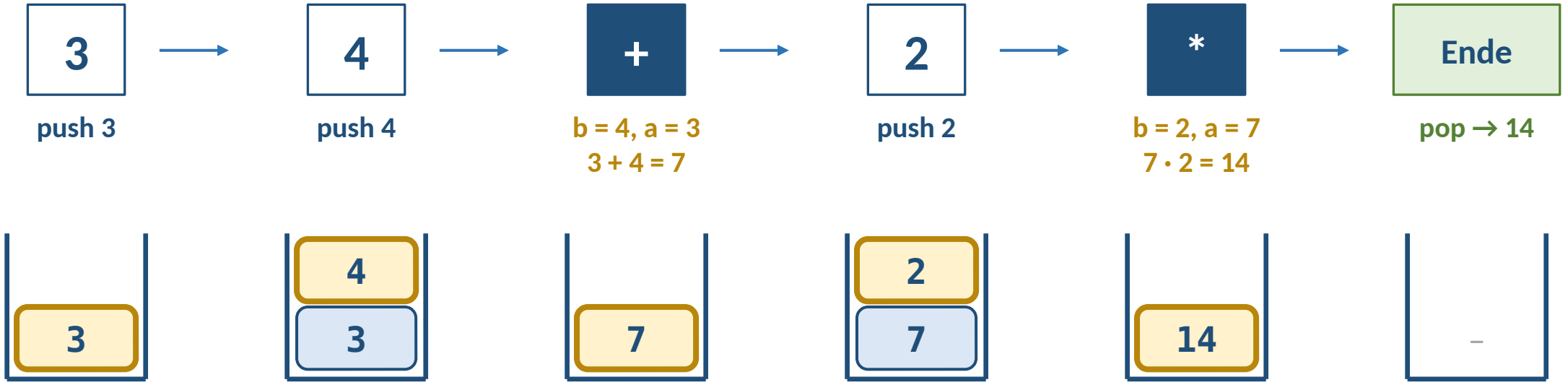
"())"

zweite): Stapel ist leer → false. Ohne Leer-Abfrage: Laufzeitfehler!

Ende

Stapel nicht leer: eine Klammer bleibt offen → false

Postfix-Rechner mit Stapel: "34+2*" Schritt für Schritt



Postfix: Der Operator steht hinter seinen Operanden – `"34+2*"` = (3 + 4) · 2 = 14, ganz ohne Klammern.

Reihenfolge!

Zuerst entnommen = RECHTER Operand:
`"93-"` → b = 3, a = 9 → 9 − 3 = 6
(vertauscht: −6)

Ziffer → Zahl

`(int) c - (int) '0'` – denn '3' hat den ASCII-Wert 51

Jetzt du (Papier)

Werte `"2534+*-"` aus und skizziere den Stapel nach jedem Zeichen.

Nebel lüften ohne Rekursion: Schlange oder Stapel?

mit Schlange (dequeue)

	0	1	2	3	4	5	6
0							
1		1	2	4	7	9	
2		3	5		10	12	
3		6	8	11	13	14	
4							

in Wellen (Breitensuche) · max. 3 gespeichert

mit Stapel (pop)

	0	1	2	3	4	5	6
0							
1		1	2	14	13	12	
2		3	4		9	11	
3		5	6	7	8	10	
4							

erst ein Gang bis zum Ende (Tiefensuche) · max. 4

Regeln

Zahl = Reihenfolge des Markierens

markiert beim Einfügen

Nachbarn: oben, rechts, unten, links

Position als eine Ganzzahl:

`z · 1000 + s`

Gleiche Endkarte: 12 Felder "." → "+" (dazu S und Z) – verschieden ist nur die Reihenfolge.

Vertiefung: ein eigener Stapel aus Knoten

Knoten

- inhalt: Object
- naechster: Knoten

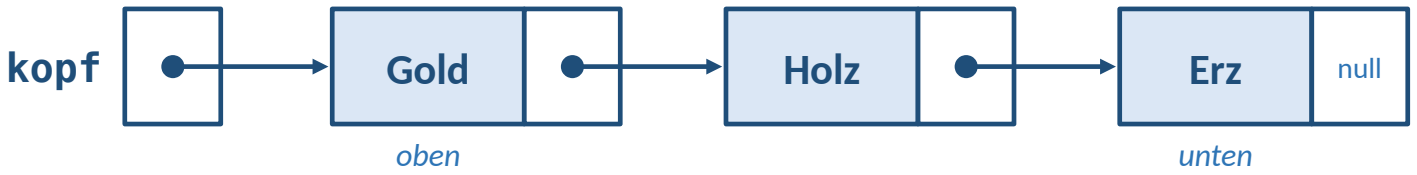
+ c Knoten(inhalt: Object, naechster: Knoten)
+ getInhalt(): Object
+ getNaechster(): Knoten
+ setInhalt(inhalt: Object): nichts
+ setNaechster(n: Knoten): nichts

MeinStapel

- kopf: Knoten

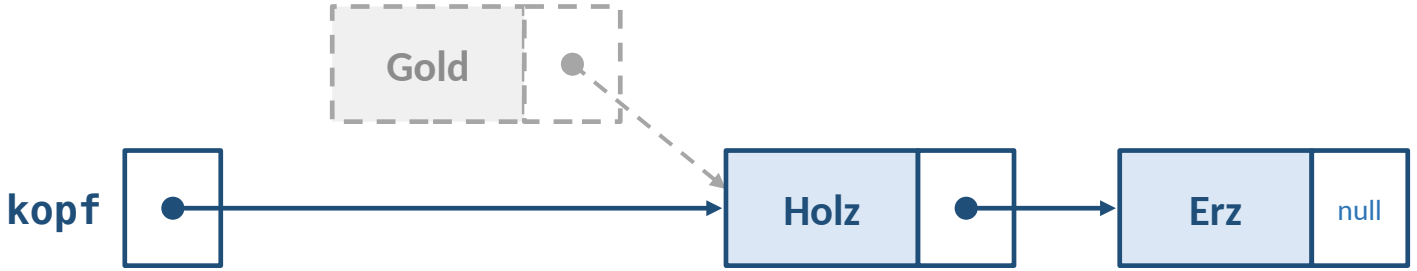
+ c MeinStapel()
+ isEmpty(): Wahrheitswert
+ push(inhalt: Object): nichts
+ top(): Object
+ pop(): Object

```
push("Erz"); push("Holz"); push("Gold");
```



pop() – erst Inhalt merken, dann kopf umhängen

liefert "Gold"



`push(x)`	neuer Knoten wird kopf, sein naechster ist der alte kopf
`pop(), top()`	Inhalt von kopf; pop hängt kopf auf den Nachfolger um
`leer`	kopf == null – top und pop werfen IllegalStateException

Abi-Format: implementieren und unverändert lassen

Aufgabe [5 BE]

Implementieren Sie die Methode `durchschnittsGewicht(q: Schlange vom Inhaltstyp Kiste): Kommazahl`. Sie liefert das durchschnittliche Gewicht der Kisten in q, bei leerer Schlange 0.0. Nach dem Aufruf muss q unverändert sein.

Abi-Check

- Leer? – kein Teilen durch 0, sondern 0.0
- Hilfsschlange und zurückschichten
- Cast beim Entnehmen: (Kiste)
- Kommazahl-Division: (double) summe / anzahl
- Rückgabe in jedem Pfad

Musterantwort

```
public static double durchschnittsGewicht(Queue q) {
    Queue hilf = new Queue();
    int summe = 0;
    int anzahl = 0;
    while (!q.isEmpty()) {
        Kiste k = (Kiste) q.dequeue();
        summe = summe + k.getGewicht();
        anzahl++;
        hilf.enqueue(k);
    }
    while (!hilf.isEmpty()) {
        q.enqueue(hilf.dequeue());
    }
    if (anzahl == 0) {
        return 0.0;            // nie durch 0 teilen
    }
    return (double) summe / anzahl;
}
```

Standard-Ladung: 33 kg / 6 = 5,5 · ganzzahlig: 5 ✕

Boss 4: Die Fähre von Nebelhafen



DS 30

60 Minuten · 24 BE
Zeitbudget wie im Abi: ≈ 2,5 min pro BE

Papier, ohne Gerät
Werkzeugkarten · ADT-Schnittstellen liegen als Hilfsmittel bei

Neuer Kontext
im Nebel vor der Hafenstadt: Stapel, Schlange und dynamische Reihung

Danach
Papier-Compiler in der IDE · zusätzlich im Lab: Online-Boss „Die Postkutschenstation“

Vorbereitung: Warm-up-Karten W4-01 bis W4-14 · Abi-Check „Leer? Index?“ · Casts mitschreiben